ChatGPT

Pipeline optimization and Person 2 metrics

Efficient data serialization

The first issue is extremely large JSON output files (~500MB). To fix this, switch to **PyTorch's binary format** (`.pt`) for saving model outputs. JSON is text-based and slow to parse, whereas `torch.save` (pickle) serializes tensors directly. For example:

```
output = {"hidden_states": model_outputs.hidden_states,
          "answer_start": answer_start_idx, "answer_end": answer_end_idx,
          "logits": model_outputs.logits,
          "input_ids": input_ids, "attention_mask": attention_mask}
torch.save(output, f"data/example_{example_id}.pt")
```

This ensures fast I/O and compact files, eliminating the high training overhead from JSON reads.

Code cleanup and structure

Remove any "dead" or unused functions from the previous code. Organize the codebase into clear modules/folders, e.g.:

- `data_loader.py` : loading `.pt` samples into PyTorch datasets.
- `metrics/cosine_drift.py` : Person 2 metrics implementation.
- `utils.py` : helper functions (if needed).
- `train_loop.py` : main training harness (if any).
- `scripts/` : CLI scripts and debugging tools.

Use **descriptive names** for files and directories. For example, store Person 1 outputs in `data/person1/` , metrics code in `metrics/` , etc. Add **progress bars** (e.g. via `tqdm`) around any data-loading or training loops to monitor progress. For example:

```
from tqdm.auto import tqdm
for batch in tqdm(dataloader, desc="Training"):
    ...
```

Also, check for MPS availability on Mac:

```
device = torch.device("mps") if torch.backends.mps.is_available() else
torch.device("cpu")
```

This ensures the code uses Apple's GPU for faster training on a MacBook Pro M4.

Cosine drift metric implementation

Implement a lean module that loads a single `.pt` sample and computes *cosine drift* over answer tokens. Key steps:

1. **Load data:** Use `torch.load("sample.pt")`. Ensure keys exist: `hidden_states`, `answer_start`, `answer_end`. If missing, raise an error ("Run Person 1 first").

2. **Hidden-state shape:** The `hidden_states` output is typically a tuple of tensors, each `(batch_size, seq_len, hidden_dim)`【turn0search1†L9-L13】. For batch size 1, slice out the answer span:

   ```
   H_ans = H_layer[0, start:end, :]
   # shape (A, hidden_dim), A = answer_end - answer_start
   ```

3. **Compute drift:** For each layer tensor `H_ans`, compute a drift array `d` of length `A`:

   ```
   import torch.nn.functional as F
   drift = torch.zeros(A)
   if A > 1:
       cos_sim = F.cosine_similarity(H_ans[1:], H_ans[:-1], dim=1, eps=1e-8)
   # see formula【turn0search0†L1-L5】
       drift[1:] = 1.0 - cos_sim
   ```

   Here `d[0]=0` by definition. Use `eps=1e-8` for numerical stability【turn0search0†L1-L5】.

4. **Layer aggregation:** Stack drifts from selected layers (e.g. last 4) into shape `(L, A)` and average across layers to get `(A,)`. This gives one drift score per answer token.

Putting it together:

```
data = torch.load("sample.pt")
hidden_states = data["hidden_states"]
start, end = data["answer_start"], data["answer_end"]
# Compute drift for each layer
drifts = []
for H in hidden_states:  # H shape: (1, seq, hidden_dim)
    H_ans = H[0, start:end, :]        # (A, hidden_dim)
    A = H_ans.size(0)
    drift = torch.zeros(A)
```

```
    if A > 1:
        cs = F.cosine_similarity(H_ans[1:], H_ans[:-1], dim=1, eps=1e-8)
        drift[1:] = 1.0 - cs
    drifts.append(drift)
drifts = torch.stack(drifts[-4:], dim=0)  # use last 4 layers
cosine_drift = drifts.mean(dim=0)         # final (A,) drift vector
```

This minimal code ensures only the necessary layers/outputs are used for prediction. Save the result or return it for further processing.

Testing and debugging

Write unit tests for the cosine drift logic:

- **Empty answer span:** If `answer_start == answer_end`, return an empty tensor (A=0).
- **Single-token answer:** If `A==1`, output `[0.0]`.
- **Multi-token answer:** Verify `cosine_drift.shape == (A,)`, and that drifting values are in [0,2] (by cosine definition).
- **Invalid indices:** If `answer_end > seq_len`, the code should `raise ValueError`.

Also create a quick debug script (e.g. `scripts/debug_cosine_drift.py`) that:

- Loads one `.pt` file.
- Prints out `seq_len`, `answer_start`, `answer_end`, `A`.
- Prints the shape of `cosine_drift` and a few drift values.
- Optionally saves the drift tensor for inspection.

Use progress bars (tqdm) around loops in training or batch processing to visualize speed, especially useful on the M4's hardware.

Model recommendations

For good AUROC in hallucination detection with minimal parameters, **use a lightweight but capable model**. For example, models like `meta-llama/Llama-3-8B` or `tiiuae/falcon-7b-instruct` provide strong performance with relatively few parameters. These models can run with 0-shot instruction (as needed for HaluEval) and still produce reasonable embeddings.

Guidelines:

- Default to a model in the 3–8B parameter range; you can always swap out via a command-line argument.
- Only use the **top few layers** for metrics, since earlier layers are less relevant and slow down training.
- Run the model in half-precision (FP16) if available to speed up computation on Mac M4.
- Keep batch sizes moderate and use multiple DataLoader workers for parallelism (if CPU-limited).
- Place any "sanity-check" assertions at the start of scripts: e.g. check that the `.pt` files exist before computation (ask the user to run Person 1 pipeline if not).

By reorganizing the code as above, and focusing only on the needed functionality, you'll avoid the prior pitfalls (gigantic JSON loads, dead code, unclear file naming) and set up an efficient, step-by-step pipeline.

````
# Codex prompt

You are working on a revised Track B pipeline. The dataset directory contains
Person 1 outputs as `.pt` files.

**Goal:** Create optimized code that loads Person 1 `.pt` data, computes Cosine
Drift over answer tokens, and is organized cleanly.

Step-by-step tasks:

1. **Convert JSON outputs to .pt**: If any JSON forward-pass files exist in
`data/`, delete or convert them. Use `torch.save` to write all Person 1 outputs
into `.pt`. Ensure new `.pt` files include: hidden_states, logits, answer_start,
answer_end, etc. If `.pt` files already exist, skip conversion.

2. **Organize codebase**: Remove unused functions. Create a metrics module:
   - In `metrics/cosine_drift.py`, implement a function
`compute_cosine_drift(data_dict, layers=-4)` that:
     - Loads `hidden_states`, `answer_start`, `answer_end` from `data_dict`.
     - Slices answer token range and computes drift as described.
     - Returns a tensor of shape `(A,)`.
   - Make sure file and function names clearly reflect purpose.

3. **Device and model setup**: In your main script or training loop:
   - Detect device (`mps` vs CPU) for Mac M4.
   - Allow choosing the model via an argument (e.g. `--model_name`).
   - Suggest a default like `"tiiuae/falcon-7b-instruct"` or another small
model.

4. **Progress bars and training loop**: Use `tqdm` for any loops (data loading,
training epochs). Example:
   ```python
   from tqdm.auto import tqdm
   for epoch in range(num_epochs):
       for batch in tqdm(train_loader, desc=f"Epoch {epoch}"):
           ...
   ```
   This helps track the long training process.

5. **Testing and debug script**: Add a script `scripts/debug_cosine_drift.py`
that:
   - Loads one sample `.pt`.
   - Calls the cosine drift function.
   - Prints: example ID, sequence length, answer range, drift tensor shape, and
````

```
first few drift values.
   - Include assertions or error messages if data is malformed.

6. **Efficiency notes**:
   - Remove any loops that parse full sequences unnecessarily.
   - Keep memory usage low by processing one example at a time or using
DataLoaders.
   - Save output tensors (e.g. drift scores) in `.pt` format for quick
evaluation later.

Do not implement any evaluation metrics (AUROC, etc.) yet. Focus on a **minimal,
well-structured pipeline** following these steps. Ensure each file/function is
actually used and documented. There should be no unnecessary code or prints
outside of debugging output.

Once implemented, run the debug script on a couple of samples to verify the
shape and values of the cosine drift vector for each example.
```

References: PyTorch model outputs (`hidden_states`) and cosine similarity are defined as `(batch, seq_len, hidden_dim)` and $d_t = 1 - \cos$ respectively【turn0search1†L9-L13】【turn0search0†L1-L5】.
